caffe2-mem

Crate in the process of being translated from C++ to Rust. Some function bodies may still be undergoing translation.

`ComputeBlobRecyclingForDag`

`compute_blob_recycling_for_dag`

These functions compute the set of blobs in a directed acyclic graph (DAG) that can be safely recycled. Given a DAG, blobs can be recycled if they are not inputs to any nodes or if all the nodes they are inputs to have already been executed. The set of blobs that can be recycled is determined by iterating over all the nodes in the DAG and updating a set of "consumed blobs" as nodes are executed. The set of blobs that can be recycled is the set of blobs in the DAG that are not in the set of consumed blobs.

`apply_assignments`

`apply_asyncif_blob_assignments`

`apply_recurrent_blob_assignments`

These functions apply assignments of blobs to a specific device or memory type. These assignments are represented as maps from blob names to devices or memory types. `apply_assignments` applies a one-time assignment of blobs to devices or memory types, while `apply_asyncif_blob_assignments` applies assignments asynchronously, in parallel with other computations. `apply_recurrent_blob_assignments` applies assignments recurrently, meaning that the assignments are applied repeatedly as long as the corresponding blobs are in use.

`can_use_blob`

`get_blob_or_mapped_blob`

`get_free_blob`

`has_key_in_map`

`has_key_in_set`

`infer_blob_size`

These functions are all used to manage the allocation and use of blobs, which are multi-dimensional arrays used to represent data in Caffe2. `can_use_blob` checks if a blob can be reused for a new computation. `get_blob_or_mapped_blob` retrieves a blob or a mapped blob, depending on whether the blob is resident in memory or mapped from disk. `get_free_blob` retrieves a free blob for use in a computation. `has_key_in_map` and `has_key_in_set` are simple utility functions to check if a key is in a map or set, respectively. `infer_blob_size` infers the size of a blob based on the dimensions of other blobs in a computation.

`optimize_inference_net`

`optimize_net`

These functions optimize a neural network for inference or training, respectively. `optimize_inference_net` optimizes a network by fusing multiple operations into a single operation, thereby reducing the number of memory accesses required. `optimize_net` optimizes a network for training by modifying the network structure or weights to improve accuracy or reduce memory consumption.

`process_op`

`run_schema_check`

These functions are used to execute individual operations in a neural network. `process_op` takes an operation and its inputs and produces its outputs. `run_schema_check` verifies that the inputs and outputs of an operation conform to the expected schema, raising an error if they do not. Both of these functions are critical for ensuring that a neural network operates correctly and produces accurate results.

In summary, `caffe2-mem` is a Rust crate that provides functions for managing the allocation and reuse of memory in a neural network. These functions are critical for optimizing the performance of a neural network and ensuring that it produces accurate results. The crate is still in the process of being translated from C++ to Rust, but many of the core functions are already available.